NamTH Video Tools – Conditional evidence (Project 168853454872)

Private command-line tool · single user (channel owner) · developer contact: truonghoainamag@gmail.com

A. OAuth flow (OAuth = Yes)

1. First run (or `--login`): the script calls `InstalledAppFlow.run_local_server()`, which opens the standard Google sign-in and consent screen in the browser for the OAuth client of project sunlit-aura-512414-u7 (app name "NamTH Video Tools").
2. Scopes requested: `youtube.upload`, `youtube` (upload tool); `youtube.readonly`, `yt-analytics.readonly` (analytics tool).
3. The channel owner chooses their own channel and clicks Allow. The token is saved only on the owner's computer (file permission 0600).
4. Before every upload the script calls `channels.list(mine=true)` and stops if the signed-in channel is not the owner's channel ID.
5. Revocation: Google Account → Security → Third-party apps with account access (myaccount.google.com/permissions) → NamTH Video Tools → Remove access. The local token file is then deleted.

B. Upload interface (use case: video uploading)

The tool has no graphical interface; it is run from the terminal. Real output of a dry run (checks the video, title ≤100 chars, description ≤5000 bytes, tags ≤500 chars, without calling the API):

```
$ python youtube_upload.py demo-video --dry-run
Video:     demo-video/video.mp4 (0.05 GB)
Tiêu đề:   Câu cá Mỹ Hoà Hưng - An Giang: kéo mỏi tay bầy cá chim
Mô tả:     40 ký tự, 3 thẻ
Chế´độ:    private
Thumbnail: demo-video/thumbnail.jpg
Chạy thư˒xong, chưa tải gì.
```

Line meanings (Vietnamese UI): Tiêu đề = Title · Mô tả = Description (characters, tags) · Chế độ = Privacy (always **private** by default) · "Chạy thử xong, chưa tải gì" = Dry run finished, nothing uploaded.

A real run then prints: the signed-in channel name → upload progress % (resumable `videos.insert`) → the new video link → "thumbnail set" (`thumbnails.set`) → a YouTube Studio link where the owner reviews the private video and publishes it manually.

C. Analytics report (use case: internal tool / analytics & reporting)

Real report from the owner's own channel NamTH Vlog (YouTube Analytics API, last 365 days, top long videos by watch time). Used only by the owner to decide which videos need a better intro; not shared or published.

Video ID	Length	Views	Minutes watched	Avg view duration	Avg % viewed
X0bWIau8mpM	35:56	510	1197	3:52	11%
FPUBudWyXoU	30:18	272	1194	4:31	15%
so_n6RU6sWU	44:40	41	769	19:14	43%
F7gZ-rbXVO8	12:02	185	485	2:38	22%
WCavjYST2kA	27:54	80	480	6:04	22%

t2YOzmIEmF0	26:08	138	376	3:21	13%
1DA67MoUCD0	31:31	85	339	4:27	14%
sgistwGF15o	20:11	112	254	2:48	14%
tn1K_XhBCJc	24:31	103	241	2:42	11%
o0YK1ne4HvU	20:11	69	232	3:31	17%

D. Complete upload script (youtube_upload.py)

Full working script, unmodified. Credentials are read from local files and are not included.

```
#!/usr/bin/env python3
"""Tải video lên YouTube từ thư mục xuất của autoedit (video.mp4, thumbnail.jpg, noi_dung_youtube.txt).

  python youtube_upload.py --login            (đăng nhập lại, chọn kênh)
  python youtube_upload.py <thư_mục_out> [--title 1] [--privacy private|unlisted|public]
                           [--publish-at 2026-10-24T19:00] [--dry-run] [--force] [--update]

Lần đầu chạy sẽ mở trình duyệt để bạn đăng nhập Google và cho phép; mã đăng nhập được lưu lại
(~/keystores/youtube_token.json) nên lần sau không phải đăng nhập nữa.
Mặc định tải lên ở chế độ Riêng tư — kiểm tra trên Studio rồi mới công khai.
"""

import argparse
import datetime as dt
import os
import random
import sys
import time

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
from googleapiclient.http import MediaFileUpload

KEYS = os.path.expanduser("~/keystores")
CLIENT_SECRET = os.path.join(KEYS, "youtube_client_secret.json")
TOKEN = os.path.join(KEYS, "youtube_token.json")
SCOPES = ["https://www.googleapis.com/auth/youtube.upload", "https://www.googleapis.com/auth/youtube"]
CATEGORY = "19"  # Du lịch & Sự kiện
TZ = dt.timezone(dt.timedelta(hours=7))
CHANNEL = "UCq7JXYTa0ZfzKWpgqi6vu7A"  # NamTH Vlog — chỉ tải lên đúng kênh này


def read_content(path):
    """Tách noi_dung_youtube.txt theo các mục '=== ... ==='."""
    sections, cur = {}, None
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.startswith("==="):
                cur = line.strip("= \n").split(" (")[0].upper()
                sections[cur] = []
            elif cur:
                sections[cur].append(line.rstrip("\n"))
    titles = [l.split(". ", 1)[1].strip() for l in sections.get("TIÊU ĐỀ", []) if ". " in l]
    desc = "\n".join(sections.get("MÔ TẢ", [])).strip()
    tags = [t.strip() for t in " ".join(sections.get("THẺ", [])).split(",") if t.strip()]
    return titles, desc, tags


def check(title, desc, tags):
    errs = []
    if not 0 < len(title) <= 100:
        errs.append(f"tiêu đề dài {len(title)} ký tự (tối đa 100)")
    if len(desc.encode()) > 5000 or "<" in desc or ">" in desc:
        errs.append("mô tả quá 5000 byte hoặc có ký tự < >")
    if sum(len(t) + (2 if " " in t else 0) for t in tags) + len(tags) - 1 > 500:
        errs.append("tổng độ dài thẻ quá 500 ký tự")
    return errs


def credentials(relogin=False):
    creds = Credentials.from_authorized_user_file(TOKEN, SCOPES) if os.path.exists(TOKEN) and not relogin
else None
```

```
    if creds and creds.valid:
        return creds
    if creds and creds.expired and creds.refresh_token and not relogin:
        creds.refresh(Request())
    else:
        if not os.path.exists(CLIENT_SECRET):
            sys.exit(f"Chưa có {CLIENT_SECRET} — tải OAuth Client (Desktop app) từ Google Cloud về đặt đúng
tên này.")
        creds = InstalledAppFlow.from_client_secrets_file(CLIENT_SECRET, SCOPES).run_local_server(port=0)
    os.makedirs(KEYS, exist_ok=True)
    fd = os.open(TOKEN, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
    with os.fdopen(fd, "w") as f:
        f.write(creds.to_json())
    return creds


def upload(yt, video, body):
    media = MediaFileUpload(video, chunksize=16 << 20, resumable=True, mimetype="video/mp4")
    req = yt.videos().insert(part="snippet,status", body=body, media_body=media, notifySubscribers=True)
    resp, tries = None, 0
    while resp is None:
        try:
            status, resp = req.next_chunk()
            if status:
                print(f"\r  đã tải {status.progress() * 100:5.1f}%", end="", flush=True)
            tries = 0
        except HttpError as e:
            if e.resp.status not in (500, 502, 503, 504) or tries >= 8:
                raise
            tries += 1
            time.sleep(min(60, 2**tries + random.random()))
        except (ConnectionError, TimeoutError, OSError):
            if tries >= 8:
                raise
            tries += 1
            time.sleep(min(60, 2**tries + random.random()))
    print("\r  đã tải 100.0%")
    return resp["id"]


def main():
    ap = argparse.ArgumentParser(description="Tải video lên YouTube")
    ap.add_argument("folder", nargs="?", default=".", help="thư mục có video.mp4, thumbnail.jpg,
noi_dung_youtube.txt")
    ap.add_argument("--title", type=int, default=1, help="chọn tiêu đề số mấy trong noi_dung_youtube.txt")
    ap.add_argument("--privacy", default="private", choices=("private", "unlisted", "public"))
    ap.add_argument("--publish-at", help="hẹn giờ công khai, giờ Việt Nam, vd 2026-10-24T19:00")
    ap.add_argument("--dry-run", action="store_true", help="chỉ kiểm tra, không tải")
    ap.add_argument("--force", action="store_true", help="tải lại dù thư mục này đã tải trước đó")
    ap.add_argument("--login", action="store_true", help="đăng nhập lại (chọn kênh khác) rồi thoát")
    ap.add_argument("--update", action="store_true", help="chỉ cập nhật tiêu đề/mô tả/thẻ cho video đã tải
(giữ nguyên chế độ)")
    ap.add_argument("--channel", default=CHANNEL, help="ID kênh được phép tải lên")
    a = ap.parse_args()
    if a.login:
        yt = build("youtube", "v3", credentials=credentials(relogin=True))
        ch = yt.channels().list(part="snippet", mine=True).execute().get("items", [])
        print("Đã đăng nhập kênh:", ", ".join(f"{c['snippet']['title']} ({c['id']})" for c in ch) or "(không
có kênh)")
        return

    video = os.path.join(a.folder, "video.mp4")
    thumb = os.path.join(a.folder, "thumbnail.jpg")
    id_file = os.path.join(a.folder, "youtube_id.txt")
    titles, desc, tags = read_content(os.path.join(a.folder, "noi_dung_youtube.txt"))
    if not 1 <= a.title <= len(titles):
        sys.exit(f"Chỉ có {len(titles)} tiêu đề")
    title = titles[a.title - 1]
    if errs := check(title, desc, tags):
        sys.exit("Lỗi nội dung: " + "; ".join(errs))
    if not os.path.exists(video):
        sys.exit(f"Không thấy {video}")
```

```
    status = {"privacyStatus": a.privacy, "selfDeclaredMadeForKids": False, "embeddable": True}
    if a.publish_at:
        when = dt.datetime.fromisoformat(a.publish_at).replace(tzinfo=TZ)
        status.update(privacyStatus="private",
publishAt=when.astimezone(dt.timezone.utc).isoformat().replace("+00:00", "Z"))
    body = {"snippet": {"title": title, "description": desc, "tags": tags, "categoryId": CATEGORY,
                        "defaultLanguage": "vi", "defaultAudioLanguage": "vi"},
            "status": status}

    print(f"Video:     {video} ({os.path.getsize(video) / 2**30:.2f} GB)")
    print(f"Tiêu đề:   {title}")
    print(f"Mô tả:     {len(desc)} ký tự, {len(tags)} thẻ")
    print(f"Chế độ:    {status['privacyStatus']}" + (f", hẹn công khai {a.publish_at} (giờ VN)" if
a.publish_at else ""))
    print(f"Thumbnail: {thumb if os.path.exists(thumb) else '(không có)'}")
    if a.update:
        if not os.path.exists(id_file):
            sys.exit("Thư mục này chưa tải video nào")
        vid = open(id_file).read().strip()
        if a.dry_run:
            print(f"Chạy thử: sẽ cập nhật https://youtu.be/{vid}")
            return
        yt = build("youtube", "v3", credentials=credentials())
        ch = yt.channels().list(part="snippet", mine=True).execute().get("items", [])
        if not any(c["id"] == a.channel for c in ch):
            sys.exit(f"Sai kênh, cần {a.channel}. Chạy lại với --login và chọn đúng kênh.")
        yt.videos().update(part="snippet", body={"id": vid, "snippet": body["snippet"]}).execute()
        print(f"Đã cập nhật: https://studio.youtube.com/video/{vid}/edit")
        return
    if os.path.exists(id_file) and not a.force:
        sys.exit(f"Thư mục này đã tải rồi: https://youtu.be/{open(id_file).read().strip()} (thêm --force để
tải lại)")
    if a.dry_run:
        print("Chạy thử xong, chưa tải gì.")
        return

    yt = build("youtube", "v3", credentials=credentials())
    ch = yt.channels().list(part="snippet", mine=True).execute().get("items", [])
    if not any(c["id"] == a.channel for c in ch):
        names = ", ".join(f"{c['snippet']['title']} ({c['id']})" for c in ch) or "(không có kênh)"
        sys.exit(f"Sai kênh: đang đăng nhập {names}, cần {a.channel}. Chạy lại với --login và chọn đúng
kênh.")
    print(f"Kênh:      {ch[0]['snippet']['title']}")
    print("Đang tải video…")
    vid = upload(yt, video, body)
    with open(id_file, "w") as f:
        f.write(vid + "\n")
    print(f"Xong: https://youtu.be/{vid}")
    if os.path.exists(thumb):
        try:
            yt.thumbnails().set(videoId=vid, media_body=MediaFileUpload(thumb,
mimetype="image/jpeg")).execute()
            print("Đã đặt thumbnail.")
        except HttpError as e:
            print(f"Chưa đặt được thumbnail ({e.resp.status}) — kênh cần xác minh số điện thoại; đặt tay trên
Studio.")
    print(f"Kiểm tra trên Studio: https://studio.youtube.com/video/{vid}/edit")


if __name__ == "__main__":
    main()
```